CSE 270 Sports Analytics

Rasch model

Narek Sahakyan, Habet Madoyan

Rasch Model

Libraries used

```
library(glmnet)
library(ggplot2)
library(Matrix)
```

Bradley-Terry and Rasch model

- In Bradley-Terry model each team is either home team or away team
- We look at how many times a team being home team one against the other team being away team, and vice versa
- Bradley-Terry helps to rank teams by their ability to win

Bradley-Terry and Rasch model

- In Rasch model every team is either on defense or on offense.
  - For each possession during the game, there are 5 offensive and 5 defensive
- If Bradley-Terry model helps to rank the teams, Rasch model helps to estimate the players offensive and defensive skills

Rasch model for NBA

One row is one **possession**, not one stint. The outcome is points scored on that trip.

$$E[P_i] = \alpha + \sum_{j=1}^{5} \beta_{O_{ij}} + \sum_{j=1}^{5} \delta_{D_{ij}} + \theta H_i$$

- $i = 1, \ldots, n$ possessions, $j = 1, \ldots, 5$ players on each side
- O_{ij}: jth player on **offense** on possession i
- D_{ij}: jth player on **defense** on possession i
- $H_i = 1$ if the **offense** is the home team, else 0
- P_i: points scored on that possession (0, 2, 3, ...)
- $E[P_i]$: expected points on that possession

Rasch Model

$$E[P_i] = \alpha + \sum_{j=1}^{5} \beta_{O_{ij}} + \sum_{j=1}^{5} \delta_{D_{ij}} + \theta H_i$$

- α: league baseline points per possession
- β: **offense**. Larger (more positive) is better
- δ: **defense**. Smaller (more negative) is better: that player allows fewer points
- θ: home-court boost when $H_i = 1$

APM used one dummy per player and home margin. Rasch uses **two** numbers per player, and the Y is points on this trip.

The matrix representation of the model

Independent variables

$$\mathbf{X} = \overbrace{\begin{pmatrix} X_{11} & \dots & X_{1p} \\ X_{21} & \dots & X_{2p} \\ X_{31} & \dots & X_{3p} \\ \dots & \dots & \dots \\ X_{(n-1)1} & \dots & X_{(n-1)p} \\ X_{n1} & \dots & X_{np} \end{pmatrix}}^{\text{Offense}} \overbrace{\begin{pmatrix} X_{1(p+1)} & \dots & X_{1(2p)} \\ X_{2(p+1)} & \dots & X_{2(2p)} \\ X_{3(p+1)} & \dots & X_{3(2p)} \\ \dots & \dots & \dots \\ X_{(n-1)(p+1)} & \dots & X_{(n-1)(2p)} \\ X_{n(p+1)} & \dots & X_{n(2p)} \end{pmatrix}}^{\text{Defense}} \begin{pmatrix} H_1 \\ H_2 \\ H_3 \\ \dots \\ H_{n-1} \\ H_n \end{pmatrix}_{n \times (2p+1)}$$

for $j = 1:5$

$$X_{ij} = \begin{cases} 1 & \text{if player } j \text{ is on } \textbf{offense} \text{ on possession } i \\ 0 & \text{otherwise} \end{cases}$$

for $j = p+1:2p$

$$X_{ij} = \begin{cases} 1 & \text{if player } j-p \text{ is on } \textbf{defense} \text{ on possession } i \\ 0 & \text{otherwise} \end{cases}$$

The matrix representation of the model

Dependent variable and the coefficients

$$\mathbf{y} = \begin{pmatrix} P_1 \\ P_2 \\ P_3 \\ \dots \\ P_{n-1} \\ P_n \end{pmatrix}_{n \times 1} \qquad \beta = \begin{pmatrix} \beta_1 \\ \dots \\ \beta_p \\ \delta_1 \\ \dots \\ \delta_p \\ \theta \end{pmatrix}_{p \times 1}$$

The possession file: `nba11_12`

2011-12 lockout season. One row is **one possession**. Same ten players stay on the floor until the trip ends.

```
data(nba11_12, package = "SportsAnalytics270")
```

- Offense: `O1` to `O5` (names)
- Defense: `D1` to `D5`
- `Home`: 1 if the **offense** is the home team
- `Points`: points scored on this trip
- `Date`

Three possessions, Christmas, Knicks at home

Row 1: Celtics have the ball (`Home = 0`). Row 2: Knicks have the ball (`Home = 1`).

```
nba11_12[1:3, c("O1", "O5", "D1", "D5", "Home", "Points", "Date")]
```

```
##                O1            O5              D1            D5 Home Points
## 1   Kevin Garnett   Rajon Rondo Carmelo Anthony Landry Fields    0      0
## 2 Carmelo Anthony Landry Fields   Kevin Garnett   Rajon Rondo    1      0
## 3   Kevin Garnett   Rajon Rondo Carmelo Anthony Landry Fields    0      2
##         Date
## 1 2011-12-25
## 2 2011-12-25
## 3 2011-12-25
```

Garnett is on offense in row 1 and on defense in row 2. The same five names swap sides when the other team gets the ball.

One possession, by hand

Row 1: Celtics offense, Knicks defense, `Home = 0`, `Points = 0`.

- Five Celtics names get an **offense** dummy of 1
- Five Knicks names get a **defense** dummy of 1
- Everyone else that night: 0
- Home-court column: 0, because the offense is away

The Y for this row is 0 points.

Stack the ten names

`unlist` walks `O1` through `D5` **by column**: every offensive name first, then every defensive name.

```
player <- unlist(nba11_12[, 1:10])
length(player)
```

```
## [1] 2097810
```

```
player[c(1, nrow(nba11_12) + 1)]
```

```
##            O11            O21
##  Kevin Garnett Sasha Pavlovic
## 478 Levels: A.J. Price Aaron Gray Al-Farouq Aminu Al Harrington ... Hamady Ndiaye
```

`length(player)` is $10 \times 209{,}781$: ten names per possession. The first name is row 1, `O1`. One full column later is row 1, `O2`.

How many distinct players?

```
num.p <- length(unique(player))
num.p
```

```
## [1] 478
```

478 people. Each will get **two** columns: one as `off`, one as `def`.

Tag each name as offense or defense

First $5n$ names are the `O` columns, so they get `"off"`. The last $5n$ names are the `D` columns, so they get `"def"`.

```
prefix <- rep(c("off", "def"), each = 5 * nrow(nba11_12))
tag <- paste(prefix, player)
c(names = length(player), tags = length(tag))
```

```
##   names    tags
## 2097810 2097810
```

```
head(tag, 3)
```

```
## [1] "off Kevin Garnett"   "off Carmelo Anthony" "off Kevin Garnett"
```

```
tail(tag, 3)
```

```
## [1] "def James Jones" "def Royal Ivey"  "def James Jones"
```

`paste` glues the role to the name. `head(tag, 3)` is `O1` on the first three possessions, not three teammates. Those strings become the column names of X.

One possession as ten tags

Same Christmas row. This is what the first possession contributes to X.

```
one <- nba11_12[1, ]
paste("off", unlist(one[1:5]))
```

```
## [1] "off Kevin Garnett"   "off Sasha Pavlovic"  "off Jermaine O'Neal"
## [4] "off Ray Allen"       "off Rajon Rondo"
```

```
paste("def", unlist(one[6:10]))
```

```
## [1] "def Carmelo Anthony"   "def Amar'e Stoudemire" "def Tyson Chandler"
## [4] "def Toney Douglas"     "def Landry Fields"
```

```
c(Home = one$Home, Points = one$Points)
```

```
##   Home Points
##      0      0
```

Ten 1s in that row. The rest of the 956 player-role columns stay 0.

Size of X

- Rows: 209,781 possessions
- Columns: $478 \times 2 + 1$ home $= 957$
- Entries: about 201 million. Almost all are 0. Each row has only **10 or 11** ones (ten players, plus home if $H_i = 1$)

A **sparse** matrix stores only those ones, as pairs `(row, column)`. Next: a 2-possession toy, then the same recipe on `nba11_12`.

Toy possessions

Two trips, two on offense, two on defense. Same swap as Christmas: the other team gets the ball in row 2.

```
toy <- data.frame(
  O1 = c("Rondo", "Anthony"),
  O2 = c("Allen", "Chandler"),
  D1 = c("Anthony", "Rondo"),
  D2 = c("Chandler", "Allen"),
  Home = c(0, 1),
  Points = c(0, 2),
  stringsAsFactors = FALSE
)
toy
```

```
##        O1       O2      D1       D2 Home Points
## 1   Rondo    Allen Anthony Chandler    0      0
## 2 Anthony Chandler   Rondo    Allen    1      2
```

Row 1: Rondo/Allen. `Home = 0`. Row 2: Anthony/Chandler. `Home = 1`.

Toy: unlist, then tag

Column-major: all O1, all O2, all D1, all D2. The first half of the names are offense.

```
toy_player <- unlist(toy[, 1:4], use.names = FALSE)
toy_prefix <- rep(c("off", "def"), each = 2 * nrow(toy))
toy_tag <- paste(toy_prefix, toy_player)
data.frame(slot = seq_along(toy_tag), prefix = toy_prefix,
           player = toy_player, tag = toy_tag, stringsAsFactors = FALSE)
```

```
##   slot prefix   player          tag
## 1    1    off    Rondo    off Rondo
## 2    2    off  Anthony  off Anthony
## 3    3    off    Allen    off Allen
## 4    4    off Chandler off Chandler
## 5    5    def  Anthony  def Anthony
## 6    6    def    Rondo    def Rondo
## 7    7    def Chandler def Chandler
## 8    8    def    Allen    def Allen
```

Eight tags. Each unique tag becomes one column.

Toy: factor levels are the columns

`as.numeric(factor(tag))` is the column index. Levels are **alphabetical**.

```
toy_fac <- factor(toy_tag)
data.frame(level = seq_along(levels(toy_fac)),
           column = levels(toy_fac), stringsAsFactors = FALSE)
```

```
##   level       column
## 1     1    def Allen
## 2     2  def Anthony
## 3     3 def Chandler
## 4     4    def Rondo
## 5     5    off Allen
## 6     6  off Anthony
## 7     7 off Chandler
## 8     8    off Rondo
```

`"def Allen"` is column 1, not `"off Rondo"`. That is why `coef` is later split into all `def`, then all `off`.

Toy: the index table `(i, j)`

`i` is the possession. `j` is that tag's column. `sparseMatrix(i, j)` puts a 1 at each pair.

```
toy_i <- rep(1:nrow(toy), 4)
toy_j <- as.numeric(toy_fac)
data.frame(i = toy_i, tag = toy_tag, j = toy_j, stringsAsFactors = FALSE)
```

```
##   i          tag j
## 1 1    off Rondo 8
## 2 2  off Anthony 6
## 3 1    off Allen 5
## 4 2 off Chandler 7
## 5 1  def Anthony 2
## 6 2    def Rondo 4
## 7 1 def Chandler 3
## 8 2    def Allen 1
```

Possession 1 appears four times (two `off`, two `def`). No home yet.

Toy: those ones as a dense grid

This is what the sparse object *means*. Zeros are not stored.

```
X_toy <- sparseMatrix(toy_i, toy_j)
colnames(X_toy) <- levels(toy_fac)
as.matrix(X_toy)
```

```
##      def Allen def Anthony def Chandler def Rondo off Allen off Anthony
## [1,]     FALSE         TRUE          TRUE     FALSE       TRUE        FALSE
## [2,]      TRUE        FALSE         FALSE      TRUE      FALSE         TRUE
##      off Chandler off Rondo
## [1,]         FALSE       TRUE
## [2,]          TRUE      FALSE
```

Row 1 has 1s under Rondo/Allen on offense and Anthony/Chandler on defense. Row 2 is the swap.

Toy: insert home as column 1

Shift every player column by $+1$. Possessions with `Home = 1` get an extra pair `(row, 1)`.

```
toy_i2 <- c(which(toy$Home == 1), toy_i)
toy_j2 <- c(rep(1, sum(toy$Home == 1)), toy_j + 1)
X_toy2 <- sparseMatrix(toy_i2, toy_j2)
colnames(X_toy2) <- c("Home", levels(toy_fac))
as.matrix(X_toy2)
```

```
##       Home def Allen def Anthony def Chandler def Rondo off Allen off Anthony
## [1,] FALSE     FALSE        TRUE         TRUE     FALSE      TRUE       FALSE
## [2,]  TRUE      TRUE       FALSE        FALSE      TRUE     FALSE        TRUE
##      off Chandler off Rondo
## [1,]        FALSE      TRUE
## [2,]         TRUE     FALSE
```

```
toy$Points
```

```
## [1] 0 2
```

Row 2, `Home` is 1. `y` is `Points`: 0 then 2. Real X is this recipe with 5+5 players and 209k rows.

Real X: the same `(i, j)` recipe

`i` repeats each possession number **10** times, in the same order as `unlist` (by column).

```
tag.factor <- as.factor(tag)
i <- rep(1:nrow(nba11_12), 10)
j <- as.numeric(tag.factor)
data.frame(
  i = i[1:10],
  tag = as.character(tag.factor[1:10]),
  j = j[1:10],
  stringsAsFactors = FALSE
)
```

```
##     i                 tag   j
## 1   1   off Kevin Garnett 758
## 2   2 off Carmelo Anthony 538
## 3   3   off Kevin Garnett 758
## 4   4 off Carmelo Anthony 538
## 5   5   off Kevin Garnett 758
## 6   6 off Carmelo Anthony 538
## 7   7   off Kevin Garnett 758
## 8   8 off Carmelo Anthony 538
## 9   9   off Kevin Garnett 758
## 10 10 off Carmelo Anthony 538
```

These ten pairs are **not** the ten players on possession 1. They are `01` on possessions 1–10. Possession 1's other nine names sit n, $2n$, . . . later in `i`/`j`.

Possession 1's ten ones

Row 1 of X is the ten tags from one.

```
n <- nrow(nba11_12)
idx1 <- 0:9 * n + 1
data.frame(
  i = i[idx1],
  tag = as.character(tag.factor[idx1]),
  j = j[idx1],
  stringsAsFactors = FALSE
)
```

```
##    i                   tag   j
## 1  1     off Kevin Garnett 758
## 2  1    off Sasha Pavlovic 889
## 3  1   off Jermaine O'Neal 707
## 4  1         off Ray Allen 861
## 5  1       off Rajon Rondo 856
## 6  1   def Carmelo Anthony  60
## 7  1 def Amar'e Stoudemire  10
## 8  1    def Tyson Chandler 461
## 9  1     def Toney Douglas 443
## 10 1     def Landry Fields 300
```

`i` is 1 ten times. `j` is Garnett-off, Pavlovic-off, . . . , Fields-def.

Add home, then build the real X

Column 1 = home court. `j + 1` leaves room. `which(Home == 1)` are the rows that get a 1 there.

```
i.new <- c(which(nba11_12$Home == 1), i)
j.new <- c(rep(1, sum(nba11_12$Home == 1)), j + 1)
X <- sparseMatrix(i.new, j.new)
y <- nba11_12$Points
c(rows = nrow(X), cols = ncol(X), nnz = nnzero(X),
  home_ones = sum(nba11_12$Home == 1), player_ones = length(i))
```

```
##        rows        cols         nnz   home_ones player_ones
##      209781         957     2202859      105049     2097810
```

Nonzeros $\approx$ 10 per row, plus one extra on home-offense rows. `y` lines up with the rows of X.

Check row 1 of X

Christmas, offense away, so **no** home 1. Ten player ones only.

```
hit <- which(X[1, ] != 0)
data.frame(
  column = as.integer(hit),
  name = c("Home", sort(unique(tag)))[hit],
  value = as.numeric(X[1, hit]),
  stringsAsFactors = FALSE
)
```

```
##    column                  name value
## 1      11 def Amar'e Stoudemire     1
## 2      61   def Carmelo Anthony     1
## 3     301     def Landry Fields     1
## 4     444     def Toney Douglas     1
## 5     462    def Tyson Chandler     1
## 6     708   off Jermaine O'Neal     1
## 7     759     off Kevin Garnett     1
## 8     857       off Rajon Rondo     1
## 9     862         off Ray Allen     1
## 10    890    off Sasha Pavlovic     1
```

Column 1 (`Home`) is missing, as `Home = 0`. The names match the ten tags from possession 1.

Why sparse?

A dense 209,781 $\times$ 957 table is about 200 million zeros. Ridge only needs the ones.

`sparseMatrix(i, j)` is the index table: each pair is one 1. That is the design matrix for $\sum \beta + \sum \delta + \theta H$.

Why not ordinary `lm`?

X has 957 columns. Lineups repeat, and ten players always share the same P_i. Ordinary `lm` can give huge βs to whoever was on a lucky trip (Benson in APM).

We still want $\hat{P} = \alpha + \sum \beta + \sum \delta + \theta H$. We add a **penalty** so unused players stay near 0.

Ridge in one line

Minimize

$$\sum_i (P_i - \hat{P}_i)^2 + \lambda \sum_k (\beta_k^2 + \delta_k^2)$$

- First sum: fit the possessions
- Second sum: keep player effects small
- $\lambda = 0$ is ordinary least squares
- λ large: every β and δ is pulled toward $\mathbf{0}$ (average player)
- Home θ and intercept α are usually left unpenalized

The ten people on the trip **share** the points. Nobody gets the whole P_i.

What `glmnet` is

`glmnet` fits that penalized model. One knob, confusingly also called `alpha`, chooses the **shape** of the penalty:

- `alpha = 0`: **ridge** (sum of squares). What we use. Every player keeps a small number
- `alpha = 1`: **lasso** (sum of absolute values). Many coefficients become exactly 0
- In between: elastic net

This `alpha` is **not** the intercept α in the Rasch formula.

`standardize = FALSE` because our X is already 0/1. Scaling those columns would change the meaning of β.

What the `cv` in `cv.glmnet` does

We do not pick λ by hand. Cross-validation does.

- Split the possessions into folds (default **10**)
- For each candidate λ, fit on 9 folds, predict the held-out fold
- Score: mean squared error of $\hat{P}$ vs P
- `lambda.min`: λ with the **lowest** CV error
- `lambda.1se`: a slightly larger λ (more shrinkage) whose error is still within 1 SE of the minimum

We take `s = "lambda.min"`: the best prediction on held-out possessions.

The call, argument by argument

```
cv.glmnet(X, y, alpha = 0, standardize = FALSE)
```

- `X`: sparse dummy matrix (home + off/def columns)
- `y`: points on that possession
- `alpha = 0`: ridge, not lasso
- `standardize = FALSE`: do not rescale 0/1 columns
- `set.seed(1)`: the fold ids are random; fix them so the lecture is reproducible

Then `coef(model, s = "lambda.min")` is one number per column, plus the intercept.

Fit the Rasch ridge

```
set.seed(1)
model <- cv.glmnet(X, y, alpha = 0, standardize = FALSE)
coef <- coef(model, s = "lambda.min")[, 1]
names(coef) <- c("Intercept", "HFA", sort(unique(tag)))
head(names(coef), 4)
```

```
## [1] "Intercept"      "HFA"            "def A.J. Price" "def Aaron Gray"
```

Names: intercept, home, then every `def`/`off` tag in alphabetical order. Next we split that vector into α, θ, δ, β.

Split the coefficient vector

`"def ..."` sorts before `"off ..."`, so the first 478 player names are defense.

```
alpha <- coef[1]
theta <- coef[2]
delta <- coef[2 + 1:num.p]
beta  <- coef[2 + num.p + 1:num.p]
c(alpha = unname(alpha), theta = unname(theta),
  n_def = length(delta), n_off = length(beta))
```

```
##        alpha        theta        n_def        n_off
##   0.95283589   0.01861821 478.00000000 478.00000000
```

`alpha` is baseline points per possession. `theta` is extra points when the offense is at home.

How the coefficients are found

That `cv.glmnet` fit *is* the Rasch coefficients.

- Each row of X has ten 1s (five `off`, five `def`) and maybe home
- $y_i = P_i$ is points on that trip
- Ridge + CV, as on the previous slides: share the points, shrink toward the average player
- No closed form like a plus/minus sum

When to use Rasch (this version)

Use it when you have **possessions** (who is on offense, who is on defense, points on that trip), and you want **two** skills per player.

- Lineup design: expected points if these five play these five
- Separate "does he help scoring?" from "does he stop scoring?"
- Not a substitute for box-score scoring titles or raw plus/minus game logs

If you only have stint **margin**, stay with APM. If you only have wins, stay with Bradley–Terry.

Advantages

- Splits offense and defense; APM is one number
- Y is points on **this trip**, so a stop and a score are not mixed into one margin
- Ridge handles 478 players $\times$ 2 roles; ordinary `lm` would blow up
- Lineup math is a sum: plug in ten names plus home
- Neutral-court ratings ($\alpha + \theta/2$) are comparable across home and away

Disadvantages

- Needs play-by-play with ten names **and** points per possession, not just box scores
- Small-sample players still shrink a lot; the rating is "toward average," not a scouting report
- Linear in points: a 3 and a 2 enter as numbers, not as shot quality
- No ball-handler vs screener: five offense βs are added as equals
- Coefficients move if you change λ, the season, or who is in the file
- Does not say the player is "good"; it says how this lineup's expected points moved

Offensive coefficients β

Larger β means that player adds more expected points when he is on offense, holding the other nine fixed.

```
head(sort(beta, decreasing = TRUE), n = 10)
```

```
##      off James Harden       off LeBron James     off Dirk Nowitzki
##            0.04893444             0.03834923           0.03551613
##        off Chris Paul     off Blake Griffin     off Manu Ginobili
##            0.03074925             0.03038170           0.02980103
##       off Mike Miller off Russell Westbrook      off Andre Miller
##            0.02918340             0.02915810           0.02722582
##     off Ryan Anderson
##            0.02712350
```

These are **not** points he scored. They are extra expected points on the possession.

Defensive coefficients δ

Smaller (more negative) δ is better defense: expected points **fall** when he is on that side.

```
head(sort(delta), n = 10)
```

```
##      def Taj Gibson   def Kevin Garnett  def Andre Iguodala     def Ricky Rubio
##         -0.04021855         -0.03304900         -0.02833276         -0.02826440
##      def Tim Duncan      def Marc Gasol def Thabo Sefolosha     def Ian Mahinmi
##         -0.02711243         -0.02642119         -0.02580529         -0.02437742
##  def Gerald Wallace      def Tony Allen
##         -0.02402187         -0.02399266
```

A positive δ means the offense scores *more* than average when he is defending.

One lineup, by hand

Miami on offense, Dallas on defense, **at Dallas**. So $H_i = 0$: the offense is away.

Heat	O1	O2	O3	O4	O5
	LeBron James	Norris Cole	Udonis Haslem	Juwan Howard	Shane Battier
Mavericks	D1	D2	D3	D4	D5
	Jason Terry	Shawn Marion	Delonte West	Brian Cardinal	Brandan Wright

Plug the ten names into the same formula.

The prediction is a sum

$$E[P_i] = \alpha + \sum_{j=1}^{5} \beta_{O_{ij}} + \sum_{j=1}^{5} \delta_{D_{ij}} + \theta H_i$$

$H_i = 0$ because Miami is the away offense.

$$P = \alpha + \beta_{\text{James}} + \beta_{\text{Cole}} + \beta_{\text{Haslem}} + \beta_{\text{Howard}} + \beta_{\text{Battier}}$$

$$+ \delta_{\text{Terry}} + \delta_{\text{Marion}} + \delta_{\text{West}} + \delta_{\text{Cardinal}} + \delta_{\text{Wright}} + \theta \times 0$$

Look up the ten coefficients

The names must match `names(beta)` and `names(delta)`: `"off LeBron James"`, not `"LeBron"`.

```
off <- c("off LeBron James", "off Norris Cole", "off Udonis Haslem",
         "off Juwan Howard", "off Shane Battier")
beta[off]
```

```
##  off LeBron James   off Norris Cole off Udonis Haslem  off Juwan Howard
##        0.03834923       -0.01595075       -0.01469766       -0.01213861
## off Shane Battier
##        0.01199970
```

```
def <- c("def Jason Terry", "def Shawn Marion", "def Delonte West",
         "def Brian Cardinal", "def Brandan Wright")
delta[def]
```

```
##    def Jason Terry   def Shawn Marion   def Delonte West def Brian Cardinal
##        0.004897191        0.003625852       -0.016922591       -0.002185638
## def Brandan Wright
##        0.004074041
```

Add them

Baseline, five offense βs, five defense δs, no home boost.

```
c(alpha = unname(alpha), theta = unname(theta))
```

```
##      alpha      theta
## 0.95283589 0.01861821
```

```
alpha + sum(beta[off]) + sum(delta[def]) + theta * 0
```

```
## Intercept
## 0.9538867
```

About 0.95 expected points on this possession. A made 2 is 2, a miss is 0, so 0.95 is a typical NBA trip.

Scale to points per 100 possessions

$\alpha + \theta/2$ is a **neutral court** (half of home court), from Sam Powers. Multiply by 100 so the units are points per 100 possessions.

```
def.rating <- 100 * (alpha + theta / 2 + delta)
off.rating <- 100 * (alpha + theta / 2 + beta)
```

`delta` (or `beta`) for an average player is near 0 after ridge, so the rating is about $100(\alpha + \theta/2)$.

Best defensive ratings

If you put this player with **four average teammates**, against **five average offense**, how many points per 100 possessions would they **allow**? Lower is better.

```
head(sort(def.rating), n = 10)
```

```
##      def Taj Gibson   def Kevin Garnett  def Andre Iguodala     def Ricky Rubio
##            92.19264           92.90960           93.38122           93.38806
##      def Tim Duncan      def Marc Gasol def Thabo Sefolosha     def Ian Mahinmi
##            93.50326           93.57238           93.63397           93.77676
##  def Gerald Wallace      def Tony Allen
##            93.81231           93.81523
```

Best offensive ratings

Same thought experiment on offense: four average teammates, five average defenders. Higher is better.

```
head(sort(off.rating, decreasing = TRUE), n = 10)
```

```
##      off James Harden       off LeBron James      off Dirk Nowitzki
##             101.10794              100.04942              99.76611
##       off Chris Paul      off Blake Griffin      off Manu Ginobili
##             99.28942              99.25267              99.19460
##      off Mike Miller off Russell Westbrook       off Andre Miller
##             99.13284              99.13031              98.93708
##     off Ryan Anderson
##             98.92685
```

Raw points per 100, no regression

`X'y / column sums` is just the average P_i while that column is 1. No adjustment for who else was on the floor.

```
pp100 <- 100 * (crossprod(X, y) / colSums(X))[, 1]
names(pp100) <- c("Home", sort(unique(tag)))
def.pp100 <- pp100[1 + 1:num.p]
off.pp100 <- pp100[1 + num.p + 1:num.p]
head(sort(off.pp100, decreasing = TRUE), n = 10)
```

```
##       off Ike Diogu   off Hamady Ndiaye     off Patty Mills   off Jerome Jordan
##            139.2857            114.2857            110.8108            107.4074
##     off Eric Maynor    off James Harden   off Nick Collison   off Manu Ginobili
##            107.1942            107.1523            107.0741            107.0286
##  off James Anderson off Jordan Hamilton
##            106.9121            106.7496
```

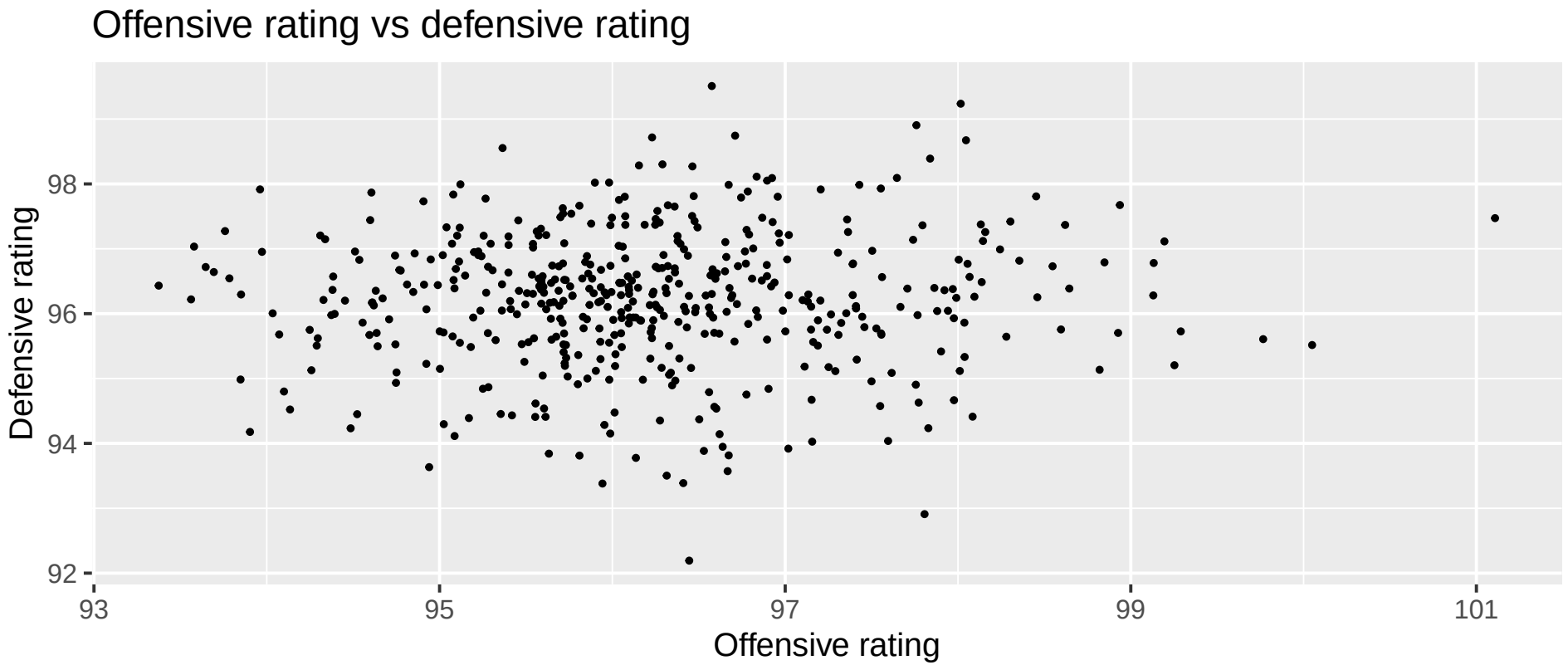
This can put a lucky bench player first, same trap as unfiltered plus/minus. The ridge ratings above are the adjusted numbers.

Offensive vs. defensive rating

Ridge β vs ridge δ, on the per-100 scale. Near 0 means no linear link: a good scorer need not be a good defender in this model.

```
cor(off.rating, def.rating)
```

```
## [1] 0.04608928
```

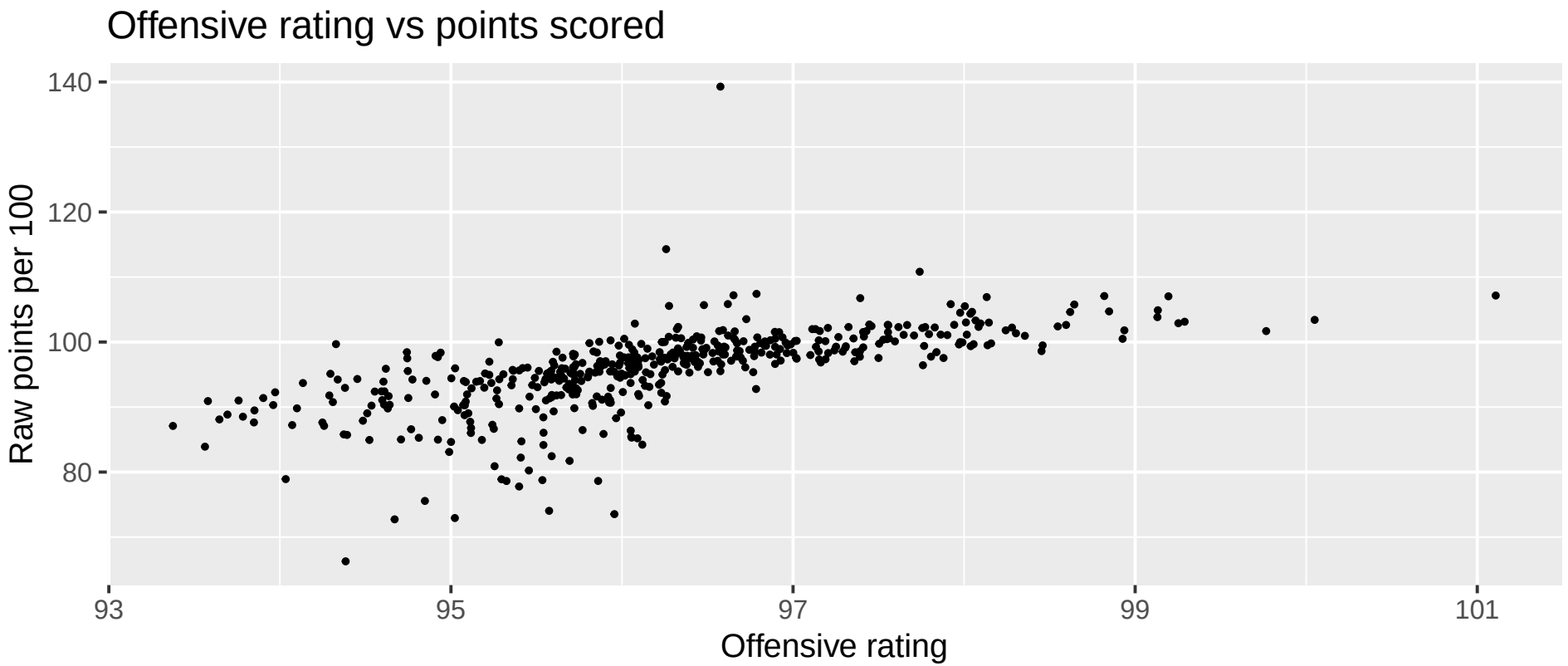


Offensive rating vs raw points

The x-axis is adjusted. The y-axis is raw points per 100 while that player was on offense.

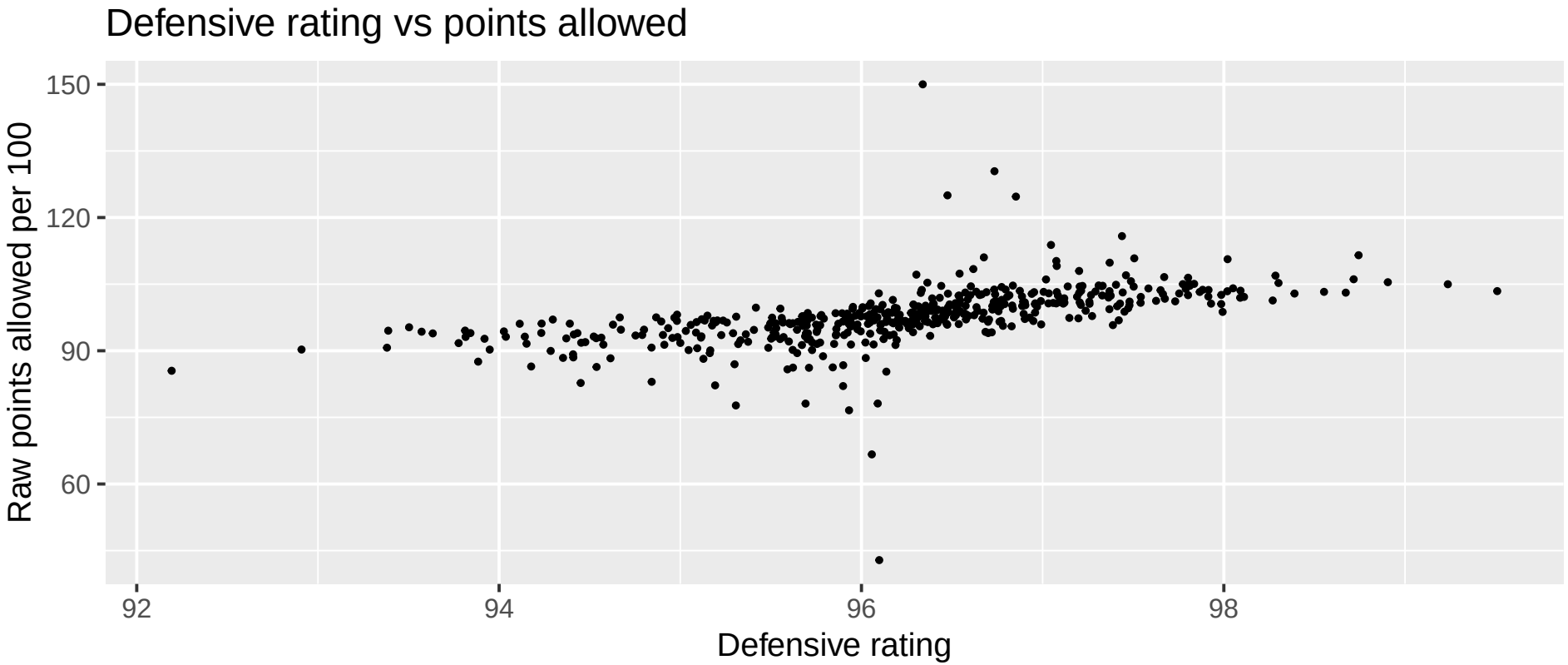
```
cor(off.rating, off.pp100)
```

```
## [1] 0.6500404
```



Defensive rating vs raw points allowed

Lower rating should go with fewer points allowed. Raw y is unadjusted.




```
## [1] 0.5430252
```

In class (10 minutes)

The fit is already in the session: `alpha`, `theta`, `beta`, `delta`, `nba11_12`. Do **not** refit `glmnet`.

Copy the next chunk. Replace every `???`. Then write one sentence under each number.

In class: fill this in

```
one <- nba11_12[1, ]
one[, c("O1", "O5", "D1", "Home", "Points")]

# 1. Dummy in this row: 1 on offense, 1 on defense, or 0.
rondo_dummy <- ???      # Rajon Rondo
carmelo_dummy <- ???    # Carmelo Anthony

# 2. Look up the matching coefficient. + raises E[P], - lowers it.
beta["off ???"]
delta["def ???"]

# 3. Expected points for possession 1. Compare to one$Points.
off1 <- paste("off", unlist(one[1:5]))
def1 <- paste("def", unlist(one[6:10]))
expected <- alpha + sum(beta[off1]) + sum(delta[def1]) + theta * ???
actual <- ???

# 4. Best offense vs best defense. Same person?
names(beta)[which.max(???)]
names(delta)[which.min(???)]
```